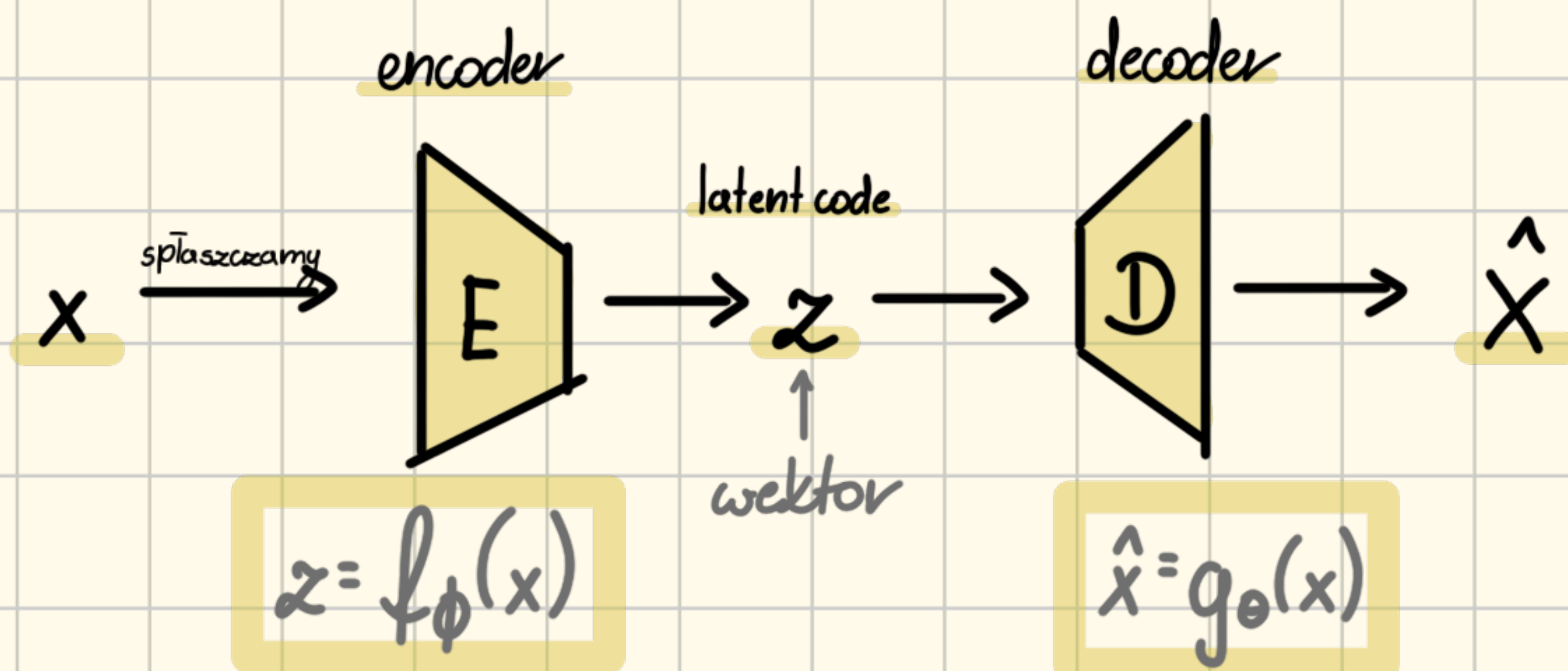


Autoencoder (AE)

- model uczenia bez nadzoru

1.



latent code = latent vector

cały model: $\hat{x} = g_{\theta}(f_{\phi}(x))$

- **encoder** bierze dane wejściowe x i kompresuje je zmieniając je na wektor z - wyciąga najważniejsze dane
- **decoder** bierze wektor z i próbuje odtworzyć dane wejściowe
- z = **latent code** to skompresowana reprezentacja danych

3. ograniczona reprezentacja latentna = z ma mniejszy rozmiar niż wejście

↳ z ma mniejszy rozmiar niż wejście więc nie może bezmyślnie kopiować

- musi nauczyć się najważniejszych informacji

trade off:

- małe z = duża kompresja → łatwa wizualizacja
- małe z = duża kompresja → gorsza rekonstrukcja
- małe z = duża kompresja → trudniejsza interpretacja
- duże z = słabsza kompresja → lepsza rekonstrukcja

- funkcja straty:

2

reconstruction loss:

= jak bardzo output różni się od inputu

$$\mathcal{L}_{AE}(x) = \|x - \hat{x}\|_2^2$$

- porównujemy x z $\hat{x}$

dla całego zbioru MSE

binary cross entropy:

$$\mathcal{L}_{AE}(\theta, \phi) = \frac{1}{n} \sum_{i=1}^n \|x_i - \hat{x}_i\|_2^2$$

$$\mathcal{L}_{BCE}(x, \hat{x}) = - \sum_{j=1}^d (x_j \log(\hat{x}_j) + (1-x_j) \log(1-\hat{x}_j))$$

- dla obrazów z pikselami $\in [0, 1]$

Jak wygląda trening?

- trenujemy jednocześnie encoder i decoder

1. mamy obrazek x

2. wsadzamy x do encodera i mamy z ($z = f_{\phi}(x)$)

3. wsadzamy z do decodera i mamy $\hat{x}$ ($\hat{x} = g_{\theta}(z)$)

4. obliczamy loss $= \|x - \hat{x}\|_2^2$

5. back propagation - aktualizacja parametrów encodera i decodera

↳ liczy gradienty, optimizer np. Adam aktualizuje parametry

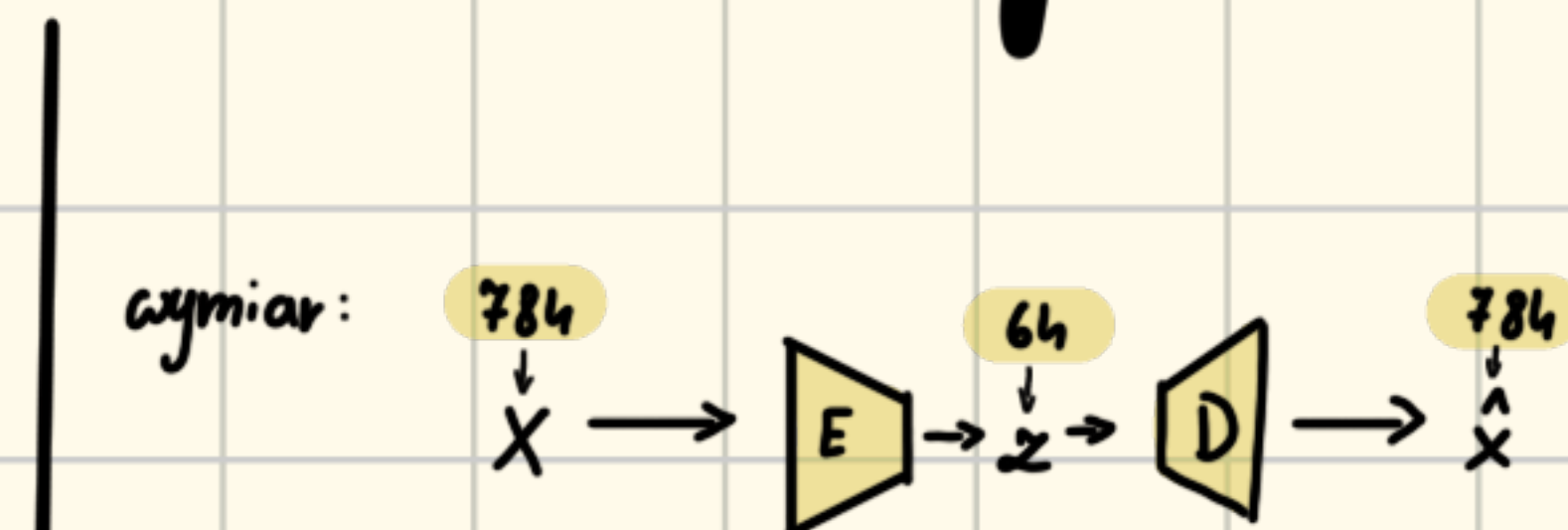
Typy AE

latent code - konkretny wektor z dla jednej próbki

latent space - cała przestrzeń możliwych wektorów z

4. Najczęściej używany typ AE

Undercomplete AE - AE gdzie wymiar z ^{latent space} jest mniejszy niż wymiar wejścia

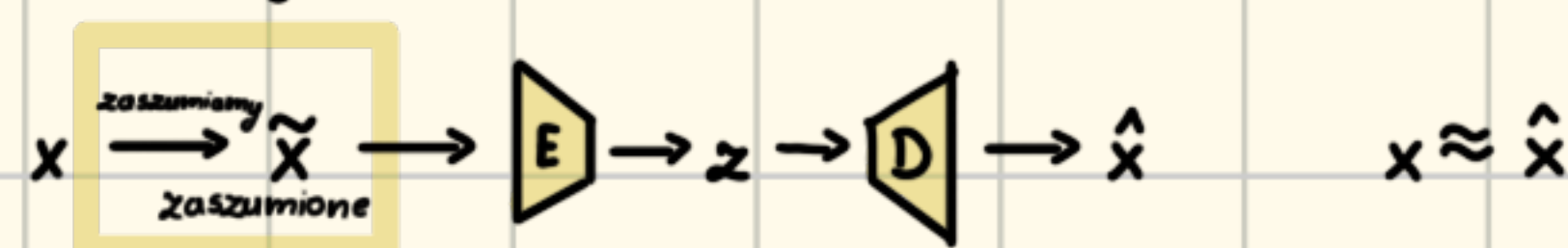


uczy się najważniejszych danych — jeśli latent space jest bardzo mały np. 2D to rekonstrukcje będą gorsze ale można łatwo zwizualizować przestrzeń latent

istnieje jeszcze Overcomplete AE, który ma wymiar 'z' większy niż wymiar wejścia - ale on może kopiować wejście więc dodaje się do niego różne ograniczenia np noise

5. Denoising AE

- dostaje na wejściu zaszumione dane i ma odtworzyć czyste dane



- zmusza to model do nauki stabilnych struktur danych

loss taki sam: $\mathcal{L}_{AE}(x) = \|x - \hat{x}\|_2^2$ - porównujemy x z $\hat{x}$

6. Sparse AE



latent może mieć wiele wymiarów ale dla danego wejścia aktywnych neuronów jest tylko mała część

- rodzaj AE w którym wymuszamy aby tylko niewiele neuronów w latent było aktywnych dla danego sampla
- dzięki temu latent może być duży i nadal zapobiega to prostemu kopiowaniu
 - dzięki temu model może nauczyć się bardzo wyspecjalizowanych cech np. określonego stylu pisma

Jak wymusza się sparsity?

normalne AE ma $\text{loss} = \mathcal{L}_{\text{AE}}$

Sparse AE $\text{loss} = \mathcal{L}_{\text{AE}} + \lambda \cdot \text{sparsity penalty}$

np. $L_1 = \|z\|_1$

- "rekonstrukcja ma być dobra ale nie używaj zbyt wielu aktywnych wartości w z "
↑
duże z
↑
zeby nie było prostego kopiowania

7.

AE w kompresji - można używać AE do kompresji danych

idea: po wytrenowaniu encodera i decodera

zamiast przechowywać x , przechowujemy z i decoderem odtwarzamy $\hat{x}$ (przybliżenie x)

czyli 1. $x \rightarrow \boxed{E} \rightarrow z$ i tak dla wszystkich danych jakie chcemy przechowywać

2. "rekonstrukcja" gdy chcemy odtworzyć: $z \rightarrow \boxed{D} \rightarrow \hat{x} \approx x$

jednak jest to kompresja stratna, czyli obraz będzie podobny ale nie identyczny ($\hat{x} \approx x \wedge \hat{x} \neq x$)

8.

AE w denoising - można za pomocą AE usłować szum z danych

no używamy denoising AE

9.

AE w anomaly detection - AE może wykrywać anomalie

idea:

1. trenujemy AE tylko na normalnych danych

↳ model uczy się dobrze rekonstruować normalne przypadki → mały loss

- gdy pojawi się nie typowy przypadek to rekonstrukcja będzie gorsza → duży loss

↓
duży reconstruction error = prawdopodobnie anomalia

anomaly score: $\text{score}(x) = \mathcal{L}_{\text{AE}} = \|x - \hat{x}\|_2^2$

trzeba jednak uważać, jeśli AE będzie "zbyt dobry" może wtedy dobrze rekonstruować anomalie → detekcja dużo słabsza

10.

AE a PCA

PCA - to klasyczna metoda redukcji wymiarowości (znajduje oś z największą wariacją dla podanych danych)

ogólne porównanie:

- PCA = liniowa kompresja
- AE = potencjalnie nieliniowa kompresja

nie z wykładu → ale jeśli w AE encoder i decoder są liniowe, AE jest Undercomplete i używamy MSE →
bez funkcji aktywacji

taki AE uczy się
podobnej przestrzeni co PCA

z wykładu! → wyniki AE rzucania danych z 3D na 2D zbiegają do wyników PCA

11.

Latent-space visualization

- można wizualizować latent-space aby zobaczyć jak model organizuje dane

- jeśli z ma 2 lub 3 wymiary to można łatwo zwizualizować jako punkt w 2D lub 3D
- jeśli z ma więcej wymiarów to można użyć np. PCA, t-SNE, UMAP do wizualizacji

wizualizacja pomaga zrozumieć czy model nauczył się sensownej struktury danych

Ograniczenia AE

normalny AE nie jest dobrym modelem generatywnym

po treningu umiemy tylko encodeować input i decodeować go

↳ nie wiadomo jak samplować sensowne nowe z

↳ niektóre punkty mogą dać realistyczny output, inne niekoniecznie

główna motywacja dla **VAE**

VAE = Variational Autoencoders

idea bardzo podobna do AE, ale oprócz rekonstrukcji danych uczy się rozkładu danych treningowych
dzięki temu VAE generuje nowe obrazy

ppb = "jak bardzo dany obrazek jest "realistyczny"/prawdopodobny"

VAE: chce nauczyć się $p_\theta(x)$ - chce maksymalizować podczas treningu - po treningu prawdziwe obrazy z data setu powinny mieć wysokie $p_\theta(x)$ a losowy sam niskie

$$p_\theta(x) = \int p_\theta(x, z) dz = \int \overset{\text{twardne}}{p_\theta(x|z)} \cdot \overset{\text{twardne bo trzeba uwzględnić wszystkie możliwe } z}{p(z)} dz$$

dlatego aproksymujemy $p_\theta(x|z)$ i $p_\theta(x)$ liczymy inaczej $\rightarrow$ ELBO

bo w ELBO i tak potrzebujemy

13a

prior to rozkład, który zakładamy dla z zanim zobaczymy dane x

$\hookrightarrow$ w VAE prawie zawsze $p(z) = \mathcal{N}(0, I)$

- prior jest rozkładem z którego losujemy nowe z żeby generować nowe dane
- i używamy go w treningu do KL loss

posterior = $p_\theta(z|x)$ = "jakie z jest prawdopodobne dla tego x ?" - aproksymujemy

13b ^{posterior}
aprobujemy $p_\theta(z|x)$

↳ podstawiamy za niego rozkład z encodera

^{rozkład z encodera}
 $q_\phi(z|x) \approx p_\theta(z|x)$

czyli encoder jest aproksymatorem posteriora

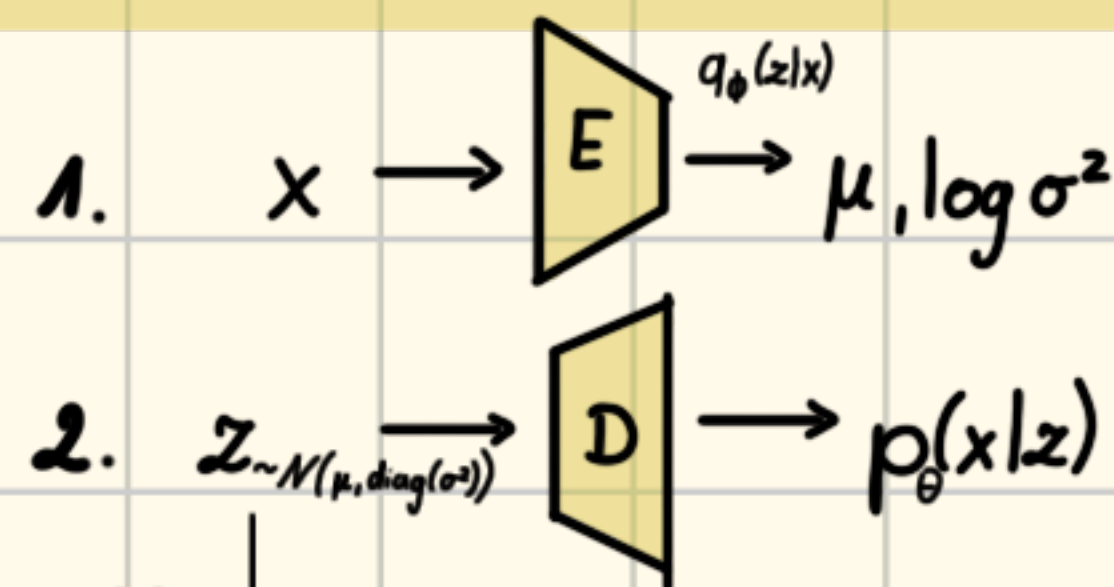
13c

↳ jak? - uczymy encoder, który na wejściu ma x i zwraca parametry rozkładu normalnego

encoder distribution: $q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \text{diag}(\sigma_\phi^2(x)))$ ← wzór tego z wykładu
↑
encoder - trenujemy to żeby dało nam dobre parametry μ, σ^2

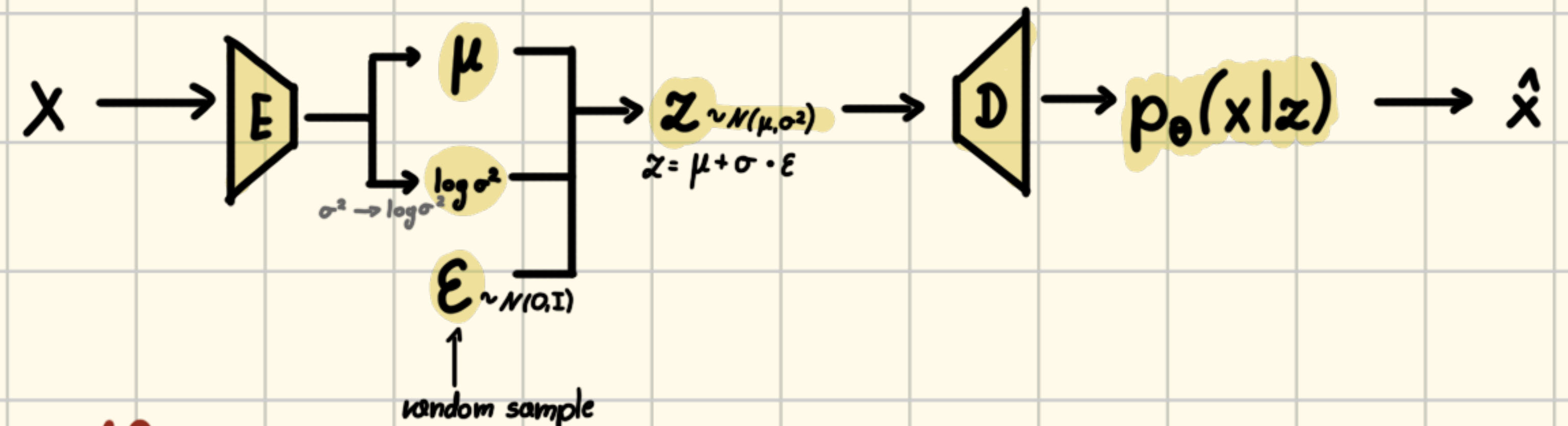
$\mu_\phi(x)$ i $\log \sigma_\phi^2(x)$ - czemu z log? bo z log jest stabilne numerycznie

zatem VAE ma dwie sieci neuronowe



18.

gdyby $z \sim \mathcal{N}(\mu, \sigma^2 I)$ to powoduje to problem przy backpropagation
↓
'z' wygląda stochastycznie i blokuje normalny gradient flow
więc używamy trik: $z = \mu + \sigma \cdot \epsilon$, gdzie $\epsilon \sim \mathcal{N}(0, I)$ więc ϵ jest jedyną losową częścią
- μ przesuwca rozkład
- σ skaluje szerokość
- ϵ daje losowość



19.

reparameterization allows gradients through stochastic sampling

$z = \mu + \sigma \cdot \epsilon$ pozwala trenować encoder mimo że z jest losowe
jeśli było by $z \sim \mathcal{N}(\mu, \sigma^2 I)$ i $z \rightarrow \hat{x}$ i liczymy loss to problem w backpropagation bo z jest losowe przez co nie ma różniczkowalnej funkcji
gdy mamy $\epsilon \sim \mathcal{N}(0, I)$ i $z = \mu + \sigma \cdot \epsilon$ to tylko ϵ jest losowe i dla konkretnego ϵ z da się różniczkować

ppb że z mogło wygenerować to x

encoder służy do aproksymacji $p_\theta(z|x)$, czyli bierze x i zwraca parametry diagonalnego rozkładu normalnego ($\mu_\phi(x)$ i $\log \sigma_\phi^2(x)$)

ppb że x zostało wygenerowane z tego z

decoder służy do "modelowania" $p_\theta(x|z)$, czyli wybieramy wcześniej "typ rozkładu" $p_\theta(x|z)$ i bierzemy $z \rightarrow$ decoder zwraca $p_\theta(x|z)$ z wytrenowanymi jego parametrami

- ma dawać wysokie $p_\theta(x|z)$ dla 'z' z encodera

wymiany latent code z są traktowane jako niezależne

"

decoder likelihood

"typ rozkładu"

13d
15

decoder likelihood to rozkład danych x pod warunkiem z czyli $p_{\theta}(x|z)$
i on zależy od tego na jakich danych działamy

- Bernulli likelihood

dla pikseli $\in [0, 1]$ lub traktowanych jak wartości 0/1 (piksele jako ppb) - dobre dla np. MNIST

$$p_{\theta}(x|z) = \prod_{j=1}^d \text{Bernoulli}(x_j, \hat{x}_j) \quad - \text{decoder uczy się } \hat{x}_j \text{ czyli ppb pikseli}$$

- wtedy reconstruction loss wychodzi jako binary cross-entropy

$$\text{reconstruction loss} = -\log p_{\theta}(x|z) = -\sum_{j=1}^d (x_j \cdot \log \hat{x}_j + (1-x_j) \log (1-\hat{x}_j)) \quad \text{używane w LOSS}$$

- Gaussian likelihood

dla danych ciągłych

$$p_{\theta}(x|z) = \mathcal{N}(x; \mu_{\theta}(z), I) \quad - \text{decoder uczy się } \mu_{\theta}(z) \text{ czyli średnią rozkładu, którą zwykle traktujemy jako rekonstrukcję: } \hat{x} = \mu_{\theta}(x)$$

- wtedy reconstruction loss jest równoważny MSE

$$\text{reconstruction loss} = -\log p_{\theta}(x|z) = \text{constant} + \frac{1}{2} \|x - \mu_{\theta}(z)\|_2^2 \quad \text{używane w LOSS}$$

16.

KL divergence - używane w LOSS

KL mierzy jak bardzo encoder distribution różni się od priora

$$D_{KL}(q_{\phi}(z|x) \parallel p(z))$$

- KL term zachęca encoder żeby jego rozkład był bliski rozkładowi priora (normalnemu najczęściej)

17.

dla diagonal Gaussian - dla wzorów: $q_{\phi}(z|x) = \mathcal{N}(\mu, \text{diag}(\sigma^2))$ i $p(z) = \mathcal{N}(0, I)$

$$D_{KL}(q_{\phi}(z|x) \parallel p(z)) = \frac{1}{2} \sum_{j=1}^d (\sigma_j^2 + \mu_j^2 - 1 - \log \sigma_j^2)$$

WAŻNE

dlaczego KL term jest ważny?

bez KL VAE zachowywałby się bardziej jak AE

- decoder mógłby rekonstruować dane treningowe, ale losowe próbkowanie mogłoby dawać bezsensowne wyniki

reconstruction loss:

w teorii: $-E_{z \sim q_{\phi}(z|x)} [\log p_{\theta}(x|z)]$

ale że w praktyce losujemy jedno z
przez reparameterization trick

to: $-\log p_{\theta}(x|z)$

14.

Evidence Lower Bound

VAE jest modelem generatywnym więc chce się nauczyć rozkładu danych – czyli dla prawdziwego x dawać wysokie $p_\theta(x)$

- problemem jest obliczyć $p_\theta(x) = \int p_\theta(x|z) \cdot p(z) dz$ ← trudna całka

+ i tak chcemy maksymalizować $\log p_\theta(x)$ bo jest stabilne numerycznie

↑
dalej ciężkie do obliczenia

więc maksymalizujemy ELBO czyli dolne ograniczenie $\log p_\theta(x)$: $\log p_\theta(x) \geq E_{z \sim q_\phi(z|x)} [\log p_\theta(x|z)] - D_{KL} = \text{ELBO}$

ELBO = reconstruction quality - KL penalty $= -E_{z \sim q_\phi(z|x)} [\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) \parallel p(z))$

- ELBO mówi czy dobrze rekonstruujemy x i czy z jest blisko priory (często $N(0, I)$)

LOSS

LOSS = -ELBO = -reconstruction quality + KL penalty = reconstruction loss + KL loss

reconstruction loss = reconstruction quality

KL loss = KL penalty

Jak trenować VAE?

dla każdego mini batcha:

1. $X \rightarrow \boxed{E} \rightarrow \mu, \log \sigma^2$
2. samplujemy $\epsilon \sim \mathcal{N}(0, I)$
3. liczymy $z = \mu + \sigma \cdot \epsilon$
4. $z \rightarrow \boxed{D} \rightarrow p_\theta(x|z) \rightarrow \hat{x}$
5. liczymy reconstruction loss = $-\log p_\theta(x|z)$ - zależy od rodzaju danych (decoder likelihood)
6. liczymy KL loss: D_{KL} = kara za oddalanie się rozkładu encodera od priory
7. minimalizujemy

$$LOSS = -ELBO = \text{reconstruction loss} + \text{KL loss}$$

- backpropagation aktualizuje encoder i decoder

- b) dla danych ciągłych (Gaussian) - model uczy się $\mu_\theta(z)$ średnia rozkładu normalnego $\rightarrow$ uznaje się $\hat{x} = \mu_\theta(z)$
parametry to średnia wartości pikseli

więc dla $\mu_\theta(z) = [0.7, 0.2, 0.6, 0.1] \leftrightarrow \hat{x} = [0.7, 0.2, 0.6, 0.1]$
 $\hat{x}_j$ oznacza intensywność j-tego piksela

- jeśli piksele czarno-białe $\rightarrow$ intensywność czerni
- jeśli piksele RGB $\rightarrow$ intensywność danego kanału

20.

Jak generować próbki z wytrenowanego VAE?

- nie potrzebujemy już encodera

robimy:

1. losujemy latent z priora: $z \sim p(z) = \mathcal{N}(0, I)$
2. podajemy z do decodera → właściwy generator
 $\hookrightarrow x \sim p_\theta(x|z)$

mając $p(x|z)$ z decodera możemy stworzyć $\hat{x}$

- a) dla pikseli $\in [0, 1]$ lub 0/1 (Bernulli) - model uczy się $\hat{x}_j$
parametry to ppb pikseli

więc dla $\hat{x} = [0.9, 0.1, 0.8, 0.2]$ $\hat{x}_j = P(x_j = 1|z)$ i 1 = biały
 $\hat{x}$ oznacza ppb na to że piksel jest biały
 $\hat{x}_j = 0.7 \rightarrow$ jasno szary, $\hat{x}_j = 0.3 \rightarrow$ ciemno szary itd

jeśli są to piksele 0/1 to kolor zależy od thresholda

21.

Posterior collapse - to problem w VAE, w którym decoder ignoruje z

- wtedy encoder zwraca rozkład prawie taki sam jak prior dla każdego wejścia

$$q_{\phi}(z|x) \approx p(z) \quad \text{czyli } 'z' \text{ nie niesie informacji o } x$$

najczęściej dzieje się to gdy decoder jest bardzo silny i potrafi generować dane bez korzystania z z

↳ objawy:

- KL spada prawie do zera
- rekonstrukcje mogą być nie złe
- reprezentacja latent nie zawiera sensownej informacji
- zmiana z ma mały wpływ na wynik

trik na to: **KL annealing**

$$L(x) = L_{\text{reconstruction}}(x) + \beta \cdot D_{\text{KL}}(q_{\phi}(z|x) \parallel p(z))$$

- na początku dajemy małe β i stopniowo zwiększamy do 1

dzięki temu model najpierw uczy się korzystać z z , a dopiero później mocniej porządkujemy latent space przez KL

12.

VAE vs. AE

AE:

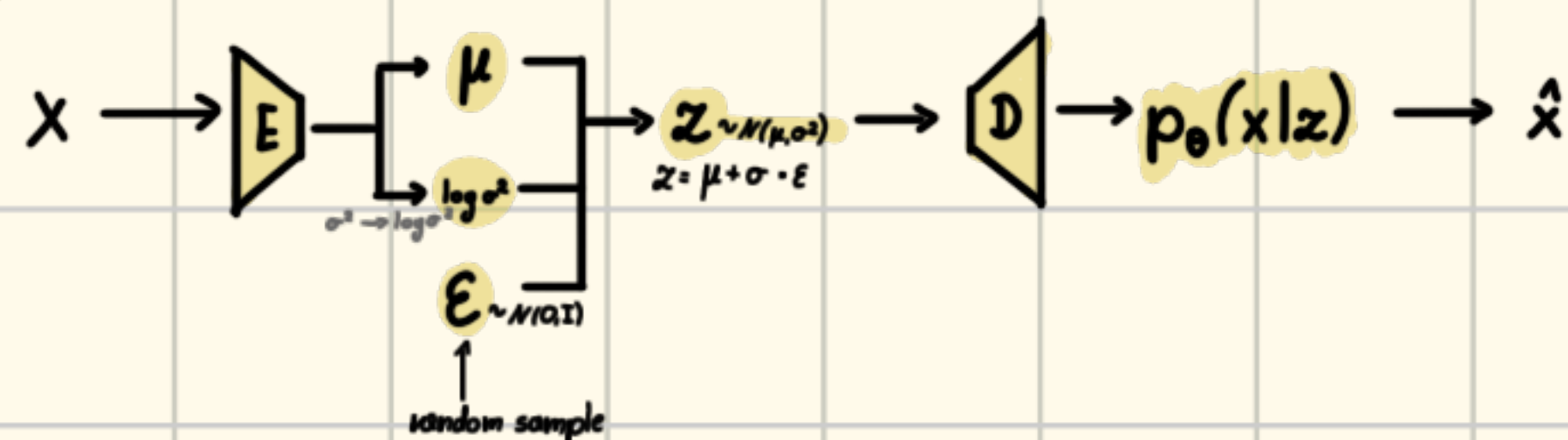
$$x \rightarrow [E] \rightarrow z \rightarrow [D] \rightarrow \hat{x} \approx x$$

AE "koduje" wejście x do jednego konkretnego latent code z , a następnie próbuje odtworzyć x

stąd zastosowania: kompresja, odszumianie, ekstrakcja cech i wykrywanie anomalii

- nie jest modelem generatywnym: AE nie wymusza konkretnego rozkładu w z więc losowe z może decodować się do bezsensownych wyników

VAE:



VAE ma podobną strukturę do AE,

ale zamiast kodować x do jednego konkretnego z , koduje je do rozkładu ppb $q_\phi(z|x)$ - opisanego parametrami μ i σ^2

funkcja straty = reconstruction loss + KL loss

KL zachęca rozkład z do bycia blisko priora $N(0,1)$

- dzięki temu po treningu można losować z i przepuszczać je przez decoder aby generować nowe dane